Web Crawler

Programming Assignment #1 — Information Retrieval

Vinicius Trindade - 2020007112
Universidade Federal de Minas Gerais
Belo Horizonte, MG, Brazil
viniciustda@ufmg.br

Abstract

This report describes a multithreaded web crawler that collects a corpus of 100,000 unique HTML pages while respecting per-site politeness constraints (`robots.txt` enforcement or a minimum per-host delay). The design combines a domain-aware frontier with round-robin scheduling, a cached `robots.txt` enforcer, and a gzip-compressed WARC writer. We discuss the main data structures and their complexity, report empirical download rates and a threads-vs-throughput speedup analysis, and characterize the crawled corpus in terms of domain dispersion and page length. The full crawl completed in 5 h 41 min at 4.88 pages/s, producing 16.7 GB of raw HTML distributed across 83,413 unique domains.

Keywords

web crawler, WARC, politeness policy, robots.txt, multithreaded I/O, corpus characterization

1 Introduction

We implemented the crawler in Python 3.14 and ran it with the command `python3 crawler.py -s <seeds> -n <limit> [-d]`, where `<seeds>` is a plain-text file with three URLs, one URL per line, and `<limit>` is the target page count (default 100,000). When `-d` is set, the crawler also prints to `stdout`, for every successfully stored page, a JSON record with the URL, page title, first 20 visible tokens, and a Unix timestamp. Internally, it initializes a frontier with URLs, fetches pages in parallel, parses them for outlinks, and stores raw HTTP responses in gzip-compressed WARC files. We applied five mandatory policies: *Selection* (HTML only, via `Content-Type: text/html`), *Revisitation* (deduplication via a normalized URL set), *Parallelization* (worker thread pool), *Politeness* (`robots.txt` `Crawl-delay` when present, 100 ms per host otherwise), and *Storage* (1,000 pages per WARC file). The code is organized into six modules: `crawler.py` (orchestration), `fetcher.py` (HTTP + robots), `frontier.py` (domain-aware queue), `parser.py` (BeautifulSoup extraction), `storage.py` (WARC writer), and `stats.py` (metrics). Section 2 details the data structures and their complexity, Section 3 reports empirical efficiency and the speedup analysis, Section 4 characterizes the final corpus, and Section 5 discloses the use of AI tools during development.

2 Data Structures and Algorithms

2.1 Frontier

The central structure maintains: (i) a dictionary $H \rightarrow \text{deque(URLs)}$ of per-host FIFO queues; (ii) a rolling host index that implements fair round-robin selection; (iii) a global hash set S of normalized URLs for deduplication; and (iv) per-host last-access timestamps, in-use flags, and `Crawl-delay` overrides for politeness enforcement. Inserting a URL costs $O(1)$ amortized (normalization, set membership, deque append). Retrieving the next URL scans at most $O(H)$ hosts to find one whose delay has elapsed and that is not currently being fetched, where H is the count of *active* hosts (i.e., hosts with a non-empty queue).

URL normalization relies on the `url-normalize` library, strips fragments, rejects non-HTTP(S) schemes, and applies a static extension blacklist (`.pdf`, `.jpg`, images, archives, fonts, …) so that non-HTML URLs never enter the frontier.

2.2 Fetcher

Parsed `robots.txt` instances (`Protego`) are cached in a dictionary keyed by host, making subsequent robots checks $O(1)$. When robots.txt specifies a `Crawl-delay`, that value replaces the default 100 ms for the corresponding host. Hosts without a declared delay keep the 100 ms default.

2.3 Parser

`BeautifulSoup` with the built-in `html.parser` extracts the title, the first 20 visible tokens (for debug output), total visible-token count, and all `<a href>` outlinks. Extraction is $O(N)$ in the size of the document.

2.4 Storage

The WARC writer opens each archive only when the first record is about to be written to it, so a run that stops exactly at a 1,000-record boundary does not leave an empty trailing archive.

2.5 Thread safety

Each shared structure (frontier, robots cache, WARC writer, stats collector) has its own mutex, implemented as a Python `threading.Lock`, so a worker blocked on one structure does not prevent access to the others. Workers run as `threading.Thread` instances. Although CPython's Global Interpreter Lock (GIL) serializes bytecode execution across threads, it is released during blocking I/O calls. Since our workload is dominated by network I/O, threads do run network requests in parallel in practice. The effective maximum limit of useful threads is set by host diversity and the 100 ms per-host delay, and is measured in Section 3: throughput plateaus between 30 and 70 workers and drops beyond that.

3 Empirical Efficiency

The main experiments ran on a laptop with an Intel Core i7-10750H (6 physical cores, 12 logical threads) on a residential broadband

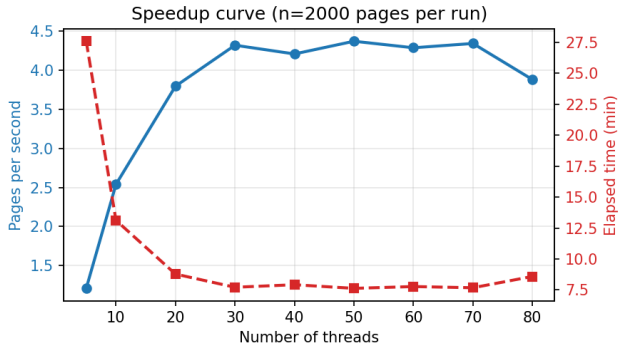


Figure 1: Pages per second (blue, solid) and wall-clock time (red, dashed) as a function of thread count on 2,000-page runs. The plateau between 30 and 70 threads marks the effective upper bound imposed by politeness and host diversity.

connection. The main crawl collected 100,000 pages in **5 h 41 min**, averaging **4.88 pages/s** and producing 16.7 GB of raw HTML.

Some experiments were also run on a Linux machine at DCC—UFMG using SSH, to validate the deployment on the department's Linux machines.

3.1 Speedup with threads

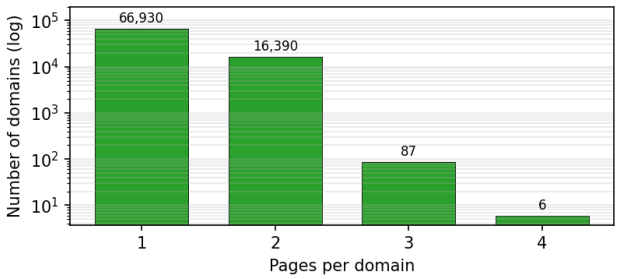
Figure 1 plots pages/s and elapsed time as a function of the thread count T, measured across 2,000-page runs with identical seeds. Throughput rises sharply up to $T = 30$, then enters a plateau between 30 and 70 threads where all runs fall within 2% of one another. At $T = 80$ throughput drops to 3.88 pages/s due to GIL contention and context-switching overhead. The plateau reflects the politeness limit: with a 100 ms delay, each host yields at most 10 pages/s, so additional threads only help when new hosts become available in the frontier.

3.2 Performance tuning

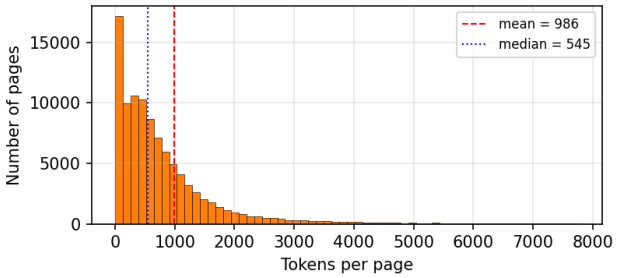
Initially, we implemented `response.apparent_encoding` on each fetched page. This function performs byte-level encoding detection using chardet's algorithm and costs 50–300 ms per response, which greatly increased the crawler's execution time. Removing it resulted in a 41% improvement in throughput (from 3.45 to 4.88 pages/s end-to-end). The removal did not negatively affect the code because `BeautifulSoup` performs its own encoding detection via `<meta charset>` inspection, and the WARC writer stores raw response bytes regardless of the decoded string encoding.

3.3 Stability over time

The instantaneous download rate (60 s sliding window) remains between 4.5 and 5.5 pages/s throughout the 5.7-hour run, without visible degradation as the frontier grows. Occasional spikes above 6 pages/s correspond to bursts of fast-responding hosts becoming simultaneously eligible for dequeue.



(a) Pages per domain.



(b) Tokens per page.

Figure 2: Corpus statistics: domain-size distribution and page-length distribution over the full 100,000-page corpus.

4 Corpus Characterization

The crawl reached **83,413 unique domains** across 100,000 pages, averaging 1.20 pages per domain, with a maximum of 4 pages for any single domain. The domain-size distribution (Fig. 2a) is extremely sparse: 66,930 domains contributed exactly one page, 16,390 contributed two, and only 93 contributed three or four. This breadth-first behavior is a direct consequence of the round-robin scheduling, which advances through all known hosts fairly: newly discovered domains enter the rotation immediately and are consumed before the crawler revisits older hosts. This is a trade-off favoring diversity over depth, a useful property for a general-purpose crawler but one that limits per-site coverage.

Page lengths follow the characteristic long-tail distribution of web content (Fig. 2b). The median page has 545 tokens and the mean 986, with a heavy right tail reaching 365,177 tokens on a handful of outliers. A non-trivial share of pages contains fewer than 200 tokens, consistent with navigation hubs, cookie walls, and error pages served as HTML responses.

The complete corpus consists of 16.7 GB of raw HTML, compressed to 3.0 GB on disk and packaged into 100 gzip-compressed WARC files (about 30 MB each). It is available on Google Drive.

5 Use of AI Tools

Claude AI was used as a development assistant for (i) plan the implementation order; (ii) review the implementation to ensure that all specification requirements were met; (iii) debug (e.g., identify `response.apparent_encoding` as a performance bottleneck); and (iv) understand some results obtained after running the simulations.